

Project Report

Comparison between PostgreSQL and Neo4j for AI Research Paper Retrieval and Knowledge Graph Analysis

Data Management 2025/2026

Last updated: May 14, 2026

1 Group Members

- Hamza Abdul Kader, matricola 2230150
- Leonardo Marasca, matricola 2217140

2 Project Goal

The goal of this project is to compare a relational DBMS and a graph database using the same AI research paper dataset. We model and analyze OpenAlex research paper data in PostgreSQL and Neo4j.

The comparison focuses on:

- data modeling
- query complexity
- expressiveness
- execution time
- suitability for relationship-heavy analysis

The project is related to Artificial Intelligence and Retrieval-Augmented Generation because the selected dataset focuses on AI, Machine Learning, Large Language Models, and RAG-related research papers.

3 Dataset

The dataset source is OpenAlex.

- Main website: <https://openalex.org/>
- Dataset documentation: OpenAlex About the data

We access the data through the OpenAlex API and collect a selected subset of research papers related to:

- retrieval augmented generation
- large language models
- artificial intelligence
- machine learning

We do not use the full OpenAlex dataset because it is very large. Instead, we collect a controlled subset that is large enough for meaningful analysis but small enough to manage in a student project.

4 Current Dataset Sample

The current sample was collected with:

```
python src/collect_openalex.py --per-term 75 --per-page 75
```

Entity	Count
Papers	299
Authors	1966
Topics	1104
Paper-author relationships	2077
Paper-topic relationships	4288
Citation relationships	392

Generated data files are stored locally under `data/raw/` and `data/processed/`. These generated files are ignored by git because they can be regenerated from the OpenAlex API.

5 Repository Structure

```
src/  
  collect_openalex.py      OpenAlex data collection and normalization  
  benchmark_queries.py     Query timing benchmark runner  
  generate_benchmark_charts.py  
  
data/raw/  
data/processed/           Generated CSV files, ignored by git  
  
database/postgres/  
database/neo4j/           PostgreSQL schema, loading script, SQL queries  
                           Neo4j constraints, import script, Cypher queries  
  
benchmarks/  
  queries.json             Paired SQL/Cypher benchmark queries  
  results/benchmark_results.csv  
  
docs/  
  project_proposal.pdf  
  project_roadmap.pdf  
  data_model.md  
  query_results.md  
  benchmark_results.md  
  project_report.md  
  project_report.pdf
```

6 Data Collection Pipeline

The script `src/collect_openalex.py` collects data from the OpenAlex Works API.

Main steps:

1. Search OpenAlex works using AI-related search terms.
2. Use cursor pagination to retrieve results.
3. Store raw API results as JSONL.
4. Normalize nested OpenAlex data into CSV files.
5. Reconstruct abstracts from OpenAlex abstract inverted indexes when available.
6. Keep only citation links where both papers are included in the selected subset.
7. Remove duplicate paper-author relationships before database loading.

The output is designed so the exact same CSV files can be loaded into both PostgreSQL and Neo4j.

7 PostgreSQL Model

PostgreSQL uses a normalized relational model.

- `papers`
- `authors`
- `topics`
- `paper_authors`
- `paper_topics`
- `citations`

Papers, authors, and topics are stored as entity tables. Many-to-many relationships are represented with bridge tables. Citations are represented as pairs of paper IDs. Indexes are created for common query patterns.

This model is strong for structured aggregation, filtering, sorting, and traditional SQL analysis.

8 Neo4j Model

Neo4j uses a graph model.

- Nodes: `Paper`, `Author`, `Topic`
- Relationships: `AUTHORED`, `HAS_TOPIC`, `CITES`

Example graph patterns:

```
(Author)-[:AUTHORED]->(Paper)
(Paper)-[:HAS_TOPIC]->(Topic)
(Paper)-[:CITES]->(Paper)
```

This model is natural for relationship-heavy questions, such as author collaboration, citation paths, shared topics, and author citation networks.

9 Docker Setup

The project uses Docker Compose to run both databases locally:

- PostgreSQL 16
- Neo4j 5

Start the databases:

```
docker compose up -d
```

Stop the databases:

```
docker compose down
```

Neo4j browser:

```
http://localhost:7474
username: neo4j
password: openalex123
```

10 Database Loading

PostgreSQL loading:

```
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/schema.sql
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/load.sql
```

Neo4j loading:

```
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 \
-f /cypher/constraints.cypher
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 \
-f /cypher/import.cypher
```

Both databases were loaded successfully with the same generated CSV files.

11 Validation

We verified that PostgreSQL and Neo4j contain matching data counts.

Data element	PostgreSQL	Neo4j
Papers	299	299
Authors	1966	1966
Topics	1104	1104
Paper-author / AUTHORED	2077	2077
Paper-topic / HAS_TOPIC	4288	4288
Citations / CITES	392	392

This confirms that the same dataset was loaded consistently into both database systems.

12 Comparison Queries

We created paired SQL and Cypher queries for the following analyses:

1. Most cited papers
2. Authors with the most papers
3. Most frequent topics
4. Author collaboration pairs
5. Citation links inside the selected subset
6. RAG-related papers by title or abstract
7. Two-hop citation paths
8. Authors connected through shared topics
9. Papers sharing cited references
10. Citation paths up to three hops
11. Author citation network

The initial query results are documented in `docs/query_results.md`. The benchmarked queries are stored in `benchmarks/queries.json`.

13 Benchmark Methodology

The benchmark script is:

```
python src/benchmark_queries.py --runs 5 --warmups 1
```

Benchmark methodology:

- one warmup run per query/system pair
- five measured runs per query/system pair
- PostgreSQL timing uses `EXPLAIN (ANALYZE, FORMAT JSON)`
- Neo4j timing uses `PROFILE` through `cypher-shell --format verbose`
- raw benchmark results are saved in `benchmarks/results/benchmark_results.csv`
- benchmark summary is saved in `docs/benchmark_results.md`

14 Benchmark Results

Query	PostgreSQL avg ms	Neo4j avg ms	Faster system
Most cited papers	0.058	4.200	PostgreSQL
Authors with the most papers	2.422	8.600	PostgreSQL
Most frequent topics	4.762	15.200	PostgreSQL
Author collaboration pairs	44.358	65.000	PostgreSQL
Citation links inside the subset	0.183	4.400	PostgreSQL
RAG-related papers	3.515	8.600	PostgreSQL
Two-hop citation paths	1.577	8.400	PostgreSQL
Authors connected through shared topics	309.447	586.600	PostgreSQL
Papers sharing cited references	2.171	7.800	PostgreSQL
Citation paths up to three hops	5.104	12.400	PostgreSQL
Author citation network	48.214	37.400	Neo4j

At the current dataset size, PostgreSQL is faster on most measured queries, especially simple ranking and aggregation queries. Neo4j is faster on the author citation network query, where the query follows a natural author-to-paper-to-paper-to-author graph pattern.

This does not mean one system is always better than the other. The dataset is still small, and performance depends on indexes, cache state, query shape, and graph density.

For this project, the strongest conclusion is that PostgreSQL is very efficient for relational aggregation, while Neo4j is often more readable and conceptually direct for graph-shaped analysis.

15 Benchmark Charts

The project also contains SVG charts generated from the benchmark CSV:

- docs/figures/benchmark_average_times.svg
- docs/figures/benchmark_relative_time.svg

They can be regenerated with:

```
python src/generate_benchmark_charts.py
```

These SVG files are useful for the final slides and for visual inspection on GitHub.

16 Technical Issue Fixed

During PostgreSQL loading, we found that OpenAlex can contain duplicate author entries for the same paper. This caused a duplicate primary key error in `paper_authors`.

Fix:

- changed the normalization logic in `src/collect_openalex.py`
- de-duplicated paper-author relationships by `(paper_id, author_id)`
- regenerated CSV files

- reloaded PostgreSQL successfully

This is useful to mention in the final presentation because it shows a realistic data cleaning issue.

17 Current Git History

```
7bb7f82 Add graph-specific benchmark queries
6bc2e6b Add benchmark visualization charts
19d3df9 Add query benchmarking results
4626f8f Add living project report
e4a7b0a Load databases and record first query results
b30eb6b Add OpenAlex data pipeline scaffold
a465622 Initialize project documentation
```

18 Current Status

Completed:

- project proposal approved
- repository created and pushed to GitHub
- project roadmap written
- OpenAlex data collector implemented
- PostgreSQL schema created
- Neo4j import model created
- Docker Compose setup created
- first real OpenAlex sample collected
- PostgreSQL loaded
- Neo4j loaded
- equivalent SQL/Cypher queries executed
- benchmark script added
- timing benchmark executed
- benchmark results documented
- benchmark charts generated
- graph-specific query pairs added

Next:

- optionally increase the dataset size and rerun benchmarks
- start shaping the final presentation structure
- write the presentation storyline and slide outline

19 Update Log

May 14, 2026

- Added four graph-specific query pairs: author shared topics, papers sharing cited references, citation paths up to three hops, and author citation network.
- Updated PostgreSQL and Neo4j query files.
- Reran the benchmark with 11 query pairs.
- Regenerated benchmark charts.
- Updated benchmark results and report interpretation.
- Added presentation-ready benchmark charts.
- Added repeatable benchmark infrastructure.
- Created the detailed living project report.
- Loaded PostgreSQL and Neo4j with the same OpenAlex CSV files.
- Verified matching counts across both systems.
- Fixed duplicate paper-author relationships in the collector.

May 12, 2026

- Project proposal was approved by Professor Roberto Maria Delfino.

May 11, 2026

- Created the project proposal PDF.
- Created the roadmap PDF.
- Created the GitHub repository documentation.